

Flow Matching

COMP4680/8650: Advanced Topics in Machine Learning

1 Introduction

The paper *Back to Basics: Let Denoising Generative Models Denoise* argues that models should predict clean data (x-prediction) rather than velocity (v-prediction). In this assignment, you will reproduce their key finding on toy 2D data, investigate whether v-prediction's failure can be overcome, and implement one-step generation using MeanFlow.

The assignment has four parts:

- **Part 1:** Verify a basic flow matching setup on 2D data.
- **Part 2:** Systematically compare prediction parameterizations and reproduce the paper's central result.
- **Part 3:** Investigate whether v-prediction's failure at high dimensions can be overcome.
- **Part 4:** Implement MeanFlow for single-step generation.

2 Setup

2.1 Flow Matching Formulation

We use flow matching with linear interpolation. The forward process is:

$$z_t = (1 - t)x + t\epsilon, \quad t \in [0, 1], \quad \epsilon \sim \mathcal{N}(0, I) \quad (1)$$

where $t = 0$ corresponds to clean data and $t = 1$ corresponds to pure noise.

v-prediction with v-loss. The simplest parameterization predicts the velocity $v = \epsilon - x$. Given a noisy sample $z_t = (1 - t)x + t\epsilon$, the training objective is:

$$\mathcal{L} = \text{MSE}(\text{model}(z_t, t), \epsilon - x) \quad (2)$$

Training procedure (for v-prediction with v-loss). Each training step:

1. Sample a batch of clean data x from the dataset.
2. Sample $t \sim \text{Uniform}(0, 1)$ and $\epsilon \sim \mathcal{N}(0, I)$.
3. Form the noisy sample $z_t = (1 - t)x + t\epsilon$.
4. Compute $\mathcal{L} = \text{MSE}(\text{model}(z_t, t), \epsilon - x)$ and update the model.

Sampling (Euler ODE). To generate samples, start from pure noise at $t = 1$ and integrate toward clean data at $t = 0$:

1. Initialize $z \sim \mathcal{N}(0, I)$ at $t = 1$.
2. For each step, compute the predicted velocity $v = \text{model}(z, t)$ and update:

$$z \leftarrow z + v \cdot \Delta t, \quad \text{where } \Delta t < 0 \text{ (stepping toward } t = 0)$$

3. The final z at $t \approx 0$ is the generated sample.

2.2 Data

We provide three 2D toy distributions: `swiss_roll`, 8-mode `gaussians`, and concentric `circles`. For each target dimension $D \in \{2, 8, 32\}$, the 2D data is multiplied by a random orthogonal matrix of shape $(2, D)$ to produce D -dimensional samples. This preserves the L_2 norm of each sample but embeds the 2D manifold into D -dimensional space. The intrinsic dimensionality is always 2, regardless of the ambient dimension D .

2.3 Provided Code and Data

You are provided with the [code repository](#), which contains:

- `src/dataloader.py`, loads precomputed projected data for each dataset and dimension.
- `pyproject.toml`, the dependency specification.
- `README.md`, setup instructions.

Everything else (model architecture, training loop, diffusion logic, sampling, evaluation scripts) will be implemented by yourself.

Data download:

```
uv run hf download xingjianleng/toy-data --local-dir data --repo-type dataset
```

2.4 Visualization

The data files include the orthogonal projection matrices P_D (shape $2 \times D$) used to create each dataset. To visualize D -dimensional samples in 2D, project back via `samples_2d = samples @ P.D.T`. The dataloader's `to_2d()` method does this for you. Use `matplotlib` to produce scatter plots.

2.5 Model Architecture

The denoiser is an MLP with the following specification:

Sinusoidal time embedding. Map the scalar $t \in [0, 1]$ to a fixed 128-dimensional embedding (reference: [DiT](#)). For embedding dimension $d = 128$, let $k = d/2 = 64$. Compute frequencies:

$$\omega_i = \exp\left(-\frac{i \cdot \ln 10000}{k-1}\right), \quad i = 0, 1, \dots, k-1$$

Then the embedding is $e_t = [\sin(t \cdot \omega_0), \dots, \sin(t \cdot \omega_{k-1}), \cos(t \cdot \omega_0), \dots, \cos(t \cdot \omega_{k-1})] \in \mathbb{R}^{128}$.

Network structure. The forward pass of `model(z, t)` is:

1. Compute the time embedding $e_t \in \mathbb{R}^{128}$ as above.

2. Concatenate: $\text{input} = [z_t; e_t] \in \mathbb{R}^{D+128}$.
3. Pass through 5 hidden layers (Linear $\rightarrow$ ReLU each, 256 units). The first maps $\mathbb{R}^{D+128} \rightarrow \mathbb{R}^{256}$; the rest map $\mathbb{R}^{256} \rightarrow \mathbb{R}^{256}$.
4. A final Linear layer (no activation) maps $\mathbb{R}^{256} \rightarrow \mathbb{R}^D$.

This is 6 Linear layers in total. The output has dimension D , and its interpretation depends on the prediction type (x or v).

MeanFlow variant (Part 4). Extend the architecture by adding a second sinusoidal embedding for the horizon $h = t - r$ (same 128-dimensional embedding, separate parameters). The input becomes $[z_t; e_t; e_h] \in \mathbb{R}^{D+256}$, and the first hidden layer maps from $\mathbb{R}^{D+256}$ to $\mathbb{R}^{256}$. The forward pass takes three arguments: `model(z, t, h)`.

2.6 A Note on Training

You are **not** expected to perfectly reproduce the ground truth distributions. Train for enough iterations so that recognizable patterns (spirals, modes, rings) emerge in your generated samples. The focus of this assignment is on understanding the *qualitative differences* between parameterizations and methods, not on achieving pixel-perfect results.

3 Part 1: Warm-up (10 marks)

3.1 Data Visualization (3 marks)

Load the `swiss_roll` dataset at $D = 2$ using the provided dataloader and visualize it with matplotlib scatter plots. Confirm you can see the spiral structure. Then load a higher-dimensional version (e.g., $D = 32$), project back to 2D using `to_2d()`, and verify that the same structure is preserved. Repeat for the `gaussians` and `circles` datasets.

What to report: Draw the original 2D dataset and the back-projected 2D dataset (from 32-dim), for each of the three datasets. So six figures in total, each worth 0.5 marks.

3.2 v-Prediction Flow Matching at $D = 2$ (7 marks)

Implement the model, training loop, and sampling procedure for v-prediction with v-loss on all 3 datasets at $D = 2$. Verify that the generated samples resemble the training distributions. At $D = 2$, all parameterizations work; this is your sanity check before scaling to higher dimensions.

The following hyperparameters are recommended:

- **Model:** as described in the Model Architecture section above
- **Optimizer:** Adam, learning rate 10^{-3}
- **Batch size:** 1024
- **Training:** 25,000 steps
- **Sampling:** Euler ODE, 50 steps (reference: [SiT](#))

What to report: Report the hyperparameters you use for flow matching training and sampling (1 mark). Include scatter plots of the generated samples for all 3 datasets at $D = 2$, alongside the

ground truth data for comparison. 2 marks for each dataset. Wrong or unexpected results will lead to mark deduction.

4 Part 2: Flow Matching Parameterization (34 marks)

Read the [JiT paper](#) (Sections 1–3). Reuse the same model, training, and sampling setup from Part 1 (5-layer MLP, 256 hidden units, Adam optimizer, 10^{-3} learning rate, 25K training steps, 50 Euler sampling steps).

1. **Calculate flow matching based on x or v -prediction (4 marks).** Building on your Part 1 implementation, the model can predict two quantities: x (clean data) or v (velocity, where $v = \epsilon - x$). The loss can be computed in either of these two spaces, giving $2 \times 2 = 4$ combinations of prediction type and loss type. Derive the conversions between parameterizations from the forward process equation $z_t = (1 - t)x + t\epsilon$: how to convert from x to v , and from v to x .

What to report: Show your derivation of the conversion formulas between prediction types.

2. **Reproduce the paper’s finding (18 marks).** Implement all 4 prediction/loss combinations. Also implement Euler ODE sampling for each prediction type. Train all 4 combinations across all 3 dataset dimensions ($D \in \{2, 8, 32\}$) and all 3 datasets. This is 36 experiments in total. Visualize the generated samples for each configuration. *Hint: if you encounter numerical instability, consider clipping t to $[\epsilon, 1 - \epsilon]$ for a suitable ϵ . With a good choice, all 4 combinations should work at $D = 2$.*

What to report: Generated sample visualizations for all configurations. Each visualization, e.g., v prediction + v loss for generated vs ground truth for **circles**, is worth 0.5 marks. 36 visualizations are worth 18 marks in total.

4.1 Questions and what to report

Consider both the training targets and the sampling procedure. Answer the following questions:

1. (3 marks) Which prediction type scales successfully to high ambient dimensions? At what dimension do the other prediction types begin to fail visibly?
2. (3 marks) Does the choice of loss space (x-loss, v-loss) affect which prediction types succeed or fail? What does this tell you about what determines generation quality?
3. (6 marks) Explain why the successful prediction type works at high D while the others do not. Consider the nature of each prediction target. *Hint: think about the rank of each prediction target relative to the ambient dimension.*

5 Part 3: Can We Rescue v-Prediction? (25 marks)

From Part 2, v -prediction fails at $D \geq 32$ with the default model and training setup. Is this failure fundamental, or can v -prediction be rescued?

Carefully read Section 3 of [RAE \(Representation Autoencoder\)](#), which discusses the relationship between parameterization and dimension. Based on your reading, propose and test approaches that might make v -prediction work at $D = 32$ on the **swiss_roll** dataset.

What to report (10 marks): Generated sample visualizations for your experiments, showing which configurations succeed and which fail. Report what works, what does not, and why.

5.1 Questions

1. (2 marks) Based on your experiments, is v-prediction's failure fundamental or can it be overcome? Do your findings support or contradict your observations from Part 2? Explain.
2. (3 marks) What approach(es) did you try? Compare the compute cost between your approach and the default x-prediction setup to achieve similar sample quality at $D = 32$.
3. (3 marks) Compare how x-prediction and v-prediction respond to your changes. Do they behave the same way? Explain why or why not. *Hint: consider what each parameterization must learn as a function of the ambient dimension D .*
4. (7 marks) In practice, v-prediction is used successfully in real image generation systems such as Stable Diffusion 3 and FLUX. Why might the situation be different for those models compared to these toy datasets?

6 Part 4: One-Step Generation (31 marks)

Parts 1 and 2 used 50-step Euler ODE integration for sampling. This section investigates whether fewer steps suffice and how to enable single-step generation.

6.1 Sampling Efficiency (3 marks)

Using your best model from Part 2, evaluate sample quality across a range of Euler step counts: 1, 2, 5, 10, 20, 50, 100, and 200 steps. Characterize how quality degrades as the number of steps decreases.

What to report: Generated sample visualizations at each step count.

6.2 MeanFlow (9 marks)

Given the limitations of few-step Euler sampling, standard flow matching generally cannot generate good samples in a single forward pass.

Read the [MeanFlow paper](#). Implement MeanFlow using **the best-performing prediction type from Part 2**. Extend the Part 1 model with an additional horizon input $h = t - r$ as described in the Model Architecture section. Use `torch.func.jvp` for the JVP computation. A flow matching ratio of 0.5 (i.e., 50% of training uses $h = 0$ for standard flow matching, 50% uses $h > 0$ for mean velocity) is a reasonable starting point. Train at $D = 32$ and compare 1-step and few-step MeanFlow against multi-step flow matching.

What to report: Side-by-side comparison of MeanFlow samples (1, 2, 5 steps) and flow matching samples (various step counts). Use all three datasets. Plot 9 figures for MeanFlow (3 datasets $\times$ 3 step counts), each worth 1 mark.

6.3 Questions

1. (2 marks) Why did you choose this prediction type for MeanFlow? Connect your answer to your findings from Part 2.

2. (4 marks) In your own words, describe the core idea behind MeanFlow. What does the model learn that is different from standard flow matching? Why does this enable one-step generation?
3. (3 marks) MeanFlow splits training between $h = 0$ (standard flow matching) and $h > 0$ (mean velocity). Why is the $h = 0$ portion needed?
4. (3 marks) Compare the training cost of MeanFlow to standard flow matching. Why is MeanFlow harder to train? What is the computational overhead of the JVP operation per training step?
5. (7 marks) Compare the MeanFlow-generated samples against the ground truth across all three datasets. Do you observe any differences or artifacts, particularly on the **gaussians** dataset? Describe what you see and explain why it occurs (or why it does not).

7 Deliverables

Your final deliverable is a **report** worth **100 marks** in total. Your report should **clearly** address the following for each part:

- **Document** your implementation approach and experiment setup.
- **Include** all requested visualizations and results.
- **Describe** your observations in your own words.
- **Answer** each question in each part according to your experiment results.

The mark distribution is: Part 1 (10), Part 2 (34), Part 3 (25), Part 4 (31). Detailed marks are listed in each section and subsection.

8 References

- [Back to Basics: Let Denoising Generative Models Denoise](#) (JiT paper on x-prediction vs v-prediction)
- [Representation Autoencoder \(RAE\)](#) (Section 3: parameterization and dimension)
- [MeanFlow: One-Step Flow Matching](#) (one-step generation via mean velocity)
- [DiT: Scalable Diffusion Models with Transformers](#) (sinusoidal time embedding reference)
- [SiT: Scalable Interpolant Transformers](#) (Euler ODE sampling reference)